

## Question 1: Employee Pay Roll

Code:

```
import java.awt.*;
import java.awt.event.*;

public class EmployeePayroll extends Frame implements
ActionListener {

    TextField idField, nameField, designationField,
salaryField, daysField;
    Choice departmentChoice;
    Checkbox r1, r2, r3, r4, r5;
    CheckboxGroup ratingGroup;
    Button generateButton, resetButton;
    TextArea reportArea;
    List reportList;

    public static void main(String[] args) {
        new EmployeePayroll();
    }

    EmployeePayroll() {
        setTitle("Employee Performance & Payroll Dashboard");
        setSize(750, 600);
        setLayout(new BorderLayout(10, 10));

        Panel inputPanel = new Panel(new GridLayout(8, 2, 8,
8));

        inputPanel.add(new Label("Employee ID:"));
        idField = new TextField();
        inputPanel.add(idField);

        inputPanel.add(new Label("Employee Name:"));
        nameField = new TextField();
        inputPanel.add(nameField);

        inputPanel.add(new Label("Department:"));
```

```
departmentChoice = new Choice();
departmentChoice.add("IT");
departmentChoice.add("HR");
departmentChoice.add("Finance");
departmentChoice.add("Marketing");
departmentChoice.add("Sales");
inputPanel.add(departmentChoice);

inputPanel.add(new Label("Designation:"));
designationField = new TextField();
inputPanel.add(designationField);

inputPanel.add(new Label("Basic Salary:"));
salaryField = new TextField();
inputPanel.add(salaryField);

inputPanel.add(new Label("Working Days:"));
daysField = new TextField();
inputPanel.add(daysField);

inputPanel.add(new Label("Performance Rating:"));

Panel ratingPanel = new Panel(new
FlowLayout(FlowLayout.LEFT));
ratingGroup = new CheckboxGroup();

r1 = new Checkbox("1", ratingGroup, false);
r2 = new Checkbox("2", ratingGroup, false);
r3 = new Checkbox("3", ratingGroup, true);
r4 = new Checkbox("4", ratingGroup, false);
r5 = new Checkbox("5", ratingGroup, false);

ratingPanel.add(r1);
ratingPanel.add(r2);
ratingPanel.add(r3);
ratingPanel.add(r4);
ratingPanel.add(r5);

inputPanel.add(ratingPanel);

Panel buttonPanel = new Panel(new FlowLayout());
generateButton = new Button("Generate Report");
```

```

        resetButton = new Button("Reset");

        generateButton.addActionListener(this);
        resetButton.addActionListener(this);

        buttonPanel.add(generateButton);
        buttonPanel.add(resetButton);

        Panel centerPanel = new Panel(new BorderLayout(10,
10));

        reportArea = new TextArea();
        reportArea.setEditable(false);

        centerPanel.add(new Label("Payroll Report"),
BorderLayout.NORTH);
        centerPanel.add(reportArea, BorderLayout.CENTER);

        reportList = new List(5);
        centerPanel.add(reportList, BorderLayout.SOUTH);

        add(inputPanel, BorderLayout.NORTH);
        add(centerPanel, BorderLayout.CENTER);
        add(buttonPanel, BorderLayout.SOUTH);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == generateButton) {
            generateReport();
        }

        if (e.getSource() == resetButton) {

```

```

        resetForm();
    }
}

void generateReport() {

    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    String designation = designationField.getText().trim();
    String department = departmentChoice.getSelectedItem();
    String ratingText =
ratingGroup.getSelectedCheckbox().getLabel();

    if (id.isEmpty() || name.isEmpty() ||
designation.isEmpty()
        || salaryField.getText().trim().isEmpty()
        || daysField.getText().trim().isEmpty()) {

        reportArea.setText("Please fill all required
fields.");
        return;
    }

    double basicSalary;
    int workingDays;
    int rating;

    try {
        basicSalary =
Double.parseDouble(salaryField.getText().trim());
        workingDays =
Integer.parseInt(daysField.getText().trim());
        rating = Integer.parseInt(ratingText);

        if (basicSalary <= 0 || workingDays <= 0 ||
workingDays > 31) {
            reportArea.setText("Enter valid salary and
working days.");
            return;
        }

    } catch (NumberFormatException ex) {

```

```

        reportArea.setText("Salary and Working Days must be
numeric.");
        return;
    }

    double grossSalary = basicSalary * workingDays / 30.0;

    double bonusRate;

    if (rating == 5) {
        bonusRate = 0.20;
    } else if (rating == 4) {
        bonusRate = 0.15;
    } else if (rating == 3) {
        bonusRate = 0.10;
    } else if (rating == 2) {
        bonusRate = 0.05;
    } else {
        bonusRate = 0.00;
    }

    double performanceBonus = grossSalary * bonusRate;
    double netSalary = grossSalary + performanceBonus;

    String report =
        "EMPLOYEE PAYROLL REPORT\n" +
        "=====\n" +
        "Employee ID      : " + id + "\n" +
        "Employee Name    : " + name + "\n" +
        "Department       : " + department + "\n" +
        "Designation      : " + designation + "\n" +
        "Basic Salary     : " + String.format("%.2f",
basicSalary) + "\n" +
        "Working Days     : " + workingDays + "\n" +
        "Performance Rating: " + rating + "\n" +
        "-----\n" +
        "Gross Salary      : " + String.format("%.2f",
grossSalary) + "\n" +
        "Performance Bonus : " + String.format("%.2f",
performanceBonus) + "\n" +
        "Net Salary       : " + String.format("%.2f",
netSalary) + "\n" +

```

```
        "=====";

        reportArea.setText(report);
        reportList.add(id + " - " + name + " - Net: " +
String.format("%.2f", netSalary));
    }

    void resetForm() {
        idField.setText("");
        nameField.setText("");
        designationField.setText("");
        salaryField.setText("");
        daysField.setText("");
        departmentChoice.select(0);
        ratingGroup.setSelectedCheckbox(r3);
        reportArea.setText("");
    }
}
```

Output:

EmployeePayroll

Employee Performance & Payroll Dashboard

Employee ID: 123

Employee Name: Lithish

Department: IT

Designation: Manager

Basic Salary: 75000

Working Days: 30

Performance Rating: ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

Payroll Report

=====

EMPLOYEE PAYROLL REPORT

=====

Employee ID : 123  
Employee Name : Lithish  
Department : IT  
Designation : Manager  
Basic Salary : 75000.00  
Working Days : 30  
Performance Rating: 5

=====

Gross Salary : 75000.00  
Performance Bonus : 15000.00  
Net Salary : 90000.00

=====

123 - Lithish - Net: 90000.00

Generate Report Reset

## Question 2: Food Ordering

```
import java.awt.*;
import java.awt.event.*;

public class FoodOrdering extends Frame implements
ActionListener {

    CardLayout cardLayout;
    Panel cards;

    TextField customerField, tableField, quantityField;

    Choice categoryChoice;
    Choice foodChoice;
```

```

TextArea orderArea;
TextArea billArea;

Button nextButton, previousButton;
Button addButton, generateButton, clearButton;

double subtotal = 0;

public static void main(String[] args) {
    new FoodOrdering();
}

FoodOrdering() {
    setTitle("Online Food Ordering and Billing System");
    setSize(700, 600);

    cardLayout = new CardLayout();
    cards = new Panel(cardLayout);

    createCustomerPanel();
    createFoodPanel();
    createBillPanel();

    add(cards);

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setLocationRelativeTo(null);
    setVisible(true);
}

void createCustomerPanel() {

    Panel panel = new Panel(new GridLayout(4, 2, 10, 10));

    panel.add(new Label("Customer Name:"));
    customerField = new TextField();
    panel.add(customerField);
}

```



```

        panel.add(new Label("Table Number:"));
        tableField = new TextField();
        panel.add(tableField);

        nextButton = new Button("Next");
        nextButton.addActionListener(this);

        panel.add(new Label(""));
        panel.add(nextButton);

        Panel main = new Panel(new BorderLayout(20, 20));

        main.add(new Label("CUSTOMER / ORDER DETAILS",
            Label.CENTER), BorderLayout.NORTH);

        main.add(panel, BorderLayout.CENTER);

        cards.add(main, "Customer");
    }

    void createFoodPanel() {

        Panel panel = new Panel(new GridLayout(5, 2, 10, 10));

        panel.add(new Label("Food Category:"));

        categoryChoice = new Choice();
        categoryChoice.add("South Indian");
        categoryChoice.add("North Indian");
        categoryChoice.add("Fast Food");
        categoryChoice.add("Chinese");

        panel.add(categoryChoice);

        panel.add(new Label("Food Item:"));

        foodChoice = new Choice();
        foodChoice.add("Dosa - 50");
        foodChoice.add("Idli - 40");
        foodChoice.add("Paneer Butter Masala - 150");
        foodChoice.add("Naan - 60");
    }

```

```
foodChoice.add("Burger - 120");
foodChoice.add("Pizza - 200");
foodChoice.add("Fried Rice - 130");
foodChoice.add("Noodles - 110");

panel.add(foodChoice);

panel.add(new Label("Quantity:"));

quantityField = new TextField();
panel.add(quantityField);

addButton = new Button("Add Item");
previousButton = new Button("Previous");
nextButton = new Button("Next");

addButton.addActionListener(this);
previousButton.addActionListener(this);
nextButton.addActionListener(this);

Panel buttons = new Panel(new FlowLayout());

buttons.add(addButton);
buttons.add(previousButton);
buttons.add(nextButton);

panel.add(new Label(""));
panel.add(buttons);

orderArea = new TextArea();
orderArea.setEditable(false);

Panel main = new Panel(new BorderLayout(10, 10));

main.add(new Label("FOOD SELECTION",
    Label.CENTER), BorderLayout.NORTH);

main.add(panel, BorderLayout.CENTER);

main.add(orderArea, BorderLayout.SOUTH);

cards.add(main, "Food");
```

```

}

void createBillPanel() {

    billArea = new TextArea();
    billArea.setEditable(false);

    generateButton = new Button("Generate Bill");
    clearButton = new Button("Clear");
    previousButton = new Button("Previous");

    generateButton.addActionListener(this);
    clearButton.addActionListener(this);
    previousButton.addActionListener(this);

    Panel buttons = new Panel(new FlowLayout());

    buttons.add(generateButton);
    buttons.add(clearButton);
    buttons.add(previousButton);

    Panel main = new Panel(new BorderLayout(10, 10));

    main.add(new Label("BILL / ORDER SUMMARY",
        Label.CENTER), BorderLayout.NORTH);

    main.add(billArea, BorderLayout.CENTER);

    main.add(buttons, BorderLayout.SOUTH);

    cards.add(main, "Bill");
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == nextButton) {

        String current = getCurrentCard();

        if (current.equals("Customer")) {

            if (customerField.getText().trim().isEmpty())

```

```

        ||
tableField.getText().trim().isEmpty()) {

        orderArea.setText("Please enter customer
name and table number.");
        cardLayout.show(cards, "Customer");
        return;
    }

    cardLayout.show(cards, "Food");

} else if (current.equals("Food")) {

    if (subtotal <= 0) {
        orderArea.setText("Please add at least one
food item.");
        return;
    }

    cardLayout.show(cards, "Bill");
}

}

if (e.getSource() == previousButton) {

    String current = getCurrentCard();

    if (current.equals("Food")) {
        cardLayout.show(cards, "Customer");
    } else if (current.equals("Bill")) {
        cardLayout.show(cards, "Food");
    }
}

if (e.getSource() == addButton) {
    addItem();
}

if (e.getSource() == generateButton) {
    generateBill();
}

```

```

        if (e.getSource() == clearButton) {
            clearOrder();
        }
    }

    void addItem() {

        if (quantityField.getText().trim().isEmpty()) {
            orderArea.setText("Please enter quantity.");
            return;
        }

        int quantity;

        try {
            quantity =
Integer.parseInt(quantityField.getText().trim());

            if (quantity <= 0) {
                orderArea.setText("Quantity must be greater
than zero.");
                return;
            }

        } catch (NumberFormatException e) {
            orderArea.setText("Quantity must be a number.");
            return;
        }

        String selectedItem = foodChoice.getSelectedItem();

        String[] parts = selectedItem.split(" - ");

        String itemName = parts[0];
        double price = Double.parseDouble(parts[1]);

        double cost = price * quantity;

        subtotal += cost;

        orderArea.append(
            itemName + " x " + quantity +

```

```

        " = Rs." + String.format("%.2f", cost) + "\n"
    );

    quantityField.setText("");
}

void generateBill() {

    double gst = subtotal * 0.05;
    double finalAmount = subtotal + gst;

    String bill =
        "ONLINE FOOD ORDER BILL\n" +
        "=====\n" +
        "Customer Name : " + customerField.getText() +
"\n" +
        "Table Number  : " + tableField.getText() +
"\n" +
        "-----\n" +
        orderArea.getText() +
        "-----\n" +
        "Subtotal      : Rs." +
        String.format("%.2f", subtotal) + "\n" +
        "GST (5%)      : Rs." +
        String.format("%.2f", gst) + "\n" +
        "-----\n" +
        "FINAL BILL    : Rs." +
        String.format("%.2f", finalAmount) + "\n" +
        "=====";

    billArea.setText(bill);
}

void clearOrder() {

    customerField.setText("");
    tableField.setText("");
    quantityField.setText("");

    orderArea.setText("");
    billArea.setText("");
}

```

```
        subtotal = 0;

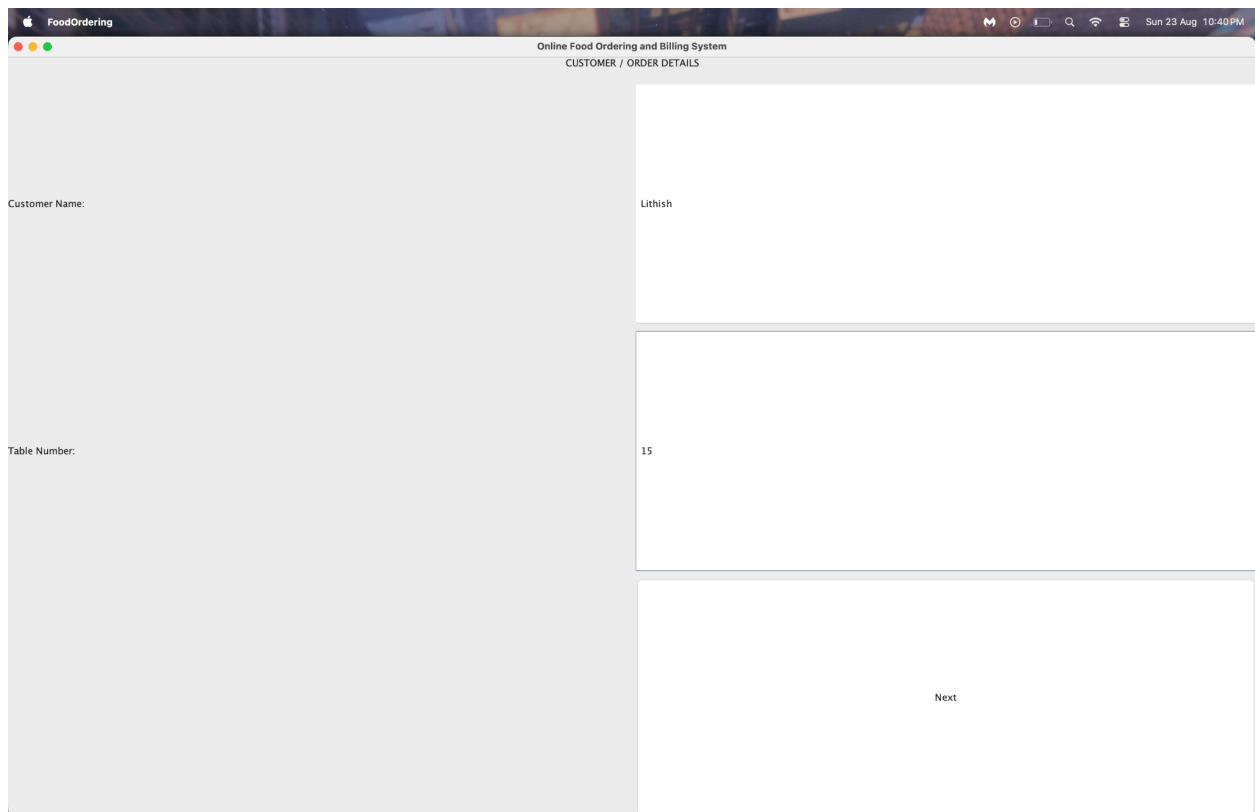
        cardLayout.show(cards, "Customer");
    }

    String getCurrentCard() {

        if (customerField.getParent() != null) {
            return "Customer";
        }

        return "Food";
    }
}
```

Output:



FoodOrdering

Online Food Ordering and Billing System

PreviousNext

Food Category:

Chinese

Food Item:

Fried Rice - 130

Quantity:

2

Add Item

FoodOrdering

Online Food Ordering and Billing System

PreviousNext

Food Category:

Chinese

Food Item:

Fried Rice - 130

Quantity:

Add Item

Fried Rice x 2 = Rs.260.00